

Examen-ProductCategory.pdf



candela_pr88



Introducción a la Ingeniería del Software y los Sistemas de Información II



2º Grado en Ingeniería Informática - Ingeniería del Software



**Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla**



[Accede al documento original](#)

 **Gemini**

**Dale al play y
estudia a tu ritmo**

Convierte estos apuntes en un track pegadizo
que me ayude a memorizar las palabras clave

Apuntes Tema 3

 PDF



Razonamiento ▾



Pruébalo



Con Selectiva.app estudia mejor, no más.

Convierte tus apuntes en recursos interactivos.



Busca en tu navegador selectiva.app

Obtén más resultados en menos tiempo

Examen DeliverUS - Octubre (ProductCategory)

Tenemos que incorporar un nuevo requisito en la aplicación DeliverUS que consiste en que las categorías de productos pueden editarse por los dueños de los restaurantes, de modo que cada restaurante tendrá sus propias categorías de productos.

1. Migraciones, modelos y cambios necesarios (2 puntos)

Cree y modifique las migraciones necesarias para implementar el modelado conceptual así como los modelos necesarios. Complete los ficheros `/src/database/migrations/20210718065004-create-product-category.js` y `/src/models/ProductCategory.js`, además de las modificaciones necesarias sobre otros ficheros.

Migraciones de productCategory:

```
id: {
  allowNull: false,
  autoIncrement: true,
  primaryKey: true,
  type: Sequelize.INTEGER
},
name: {
  allowNull: false,
  type: Sequelize.STRING
},
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
updatedAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
restaurantId: {
  allowNull: false,
  type: Sequelize.INTEGER,
  onDelete: 'CASCADE',
  references: {
    model: {
      tableName: 'Users'
    },
    key: 'id'
  }
}
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
```

WUOLAH

```

        defaultValue: new Date()
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE,
        defaultValue: new Date()
      }
    }
  }
}

```

Modelo de productCategory:

Añadimos las claves ajenas:

```

ProductCategory.hasMany(models.Product, { foreignKey: 'categoryId', as: 'products' }),
ProductCategory.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })

```

Definimos la tabla productCategory:

```

ProductCategory.init({
  name: {
    allowNull: false,
    type: Sequelize.STRING
  },
  restaurantId: {
    allowNull: false,
    type: Sequelize.INTEGER,
    onDelete: 'SET NULL',
    references: {
      model: {
        tableName: 'Users'
      },
      key: 'id'
    }
  }
})

```

Añadimos la clave ajena en restaurante:

```

Restaurant.hasMany(models.ProductCategory, { foreignKey: 'restaurantId', as: 'categories' })

```

2. Rutas de ProductCategory (2 puntos)

Implemente las siguientes rutas:

RF1: GET /productCategories/restaurants/:restaurantId

RF2: POST /productCategories/restaurants/:restaurantId

RF3: DELETE /productCategories/:restaurantId/categories/:categoryId

Se le proporciona el fichero /src/routes/ProductCategoryRoutes.js para definir estas rutas y recuerde incorporar los middlewares necesarios en cada ruta.

```

app.route('/productCategories/restaurants/:restaurantId')
  .get(
    isLoggedIn,
    hasRole('owner'),
    RestaurantMiddleware.checkRestaurantOwnership,
    checkEntityExists(Restaurant, 'restaurantId'),
    ProductCategoryController.index)
  .post(
    isLoggedIn,
    hasRole('owner'),
    RestaurantMiddleware.checkRestaurantOwnership,
    checkEntityExists(Restaurant, 'restaurantId'),
    ProductCategoryValidation.create,
    handleValidation,
    ProductCategoryController.create)

app.route('/productCategories/:restaurantId/categories/:categoryId')
  .delete(
    ProductCategoryMiddleware.checkNoProductsInCategory,
    isLoggedIn,
    hasRole('owner'),
    RestaurantMiddleware.checkRestaurantOwnership,
    checkEntityExists(Restaurant, 'restaurantId'),
    checkEntityExists(ProductCategory, 'categoryId'),
    RestaurantController.destroy)
}

```

3. Validaciones para ProductCategory (1 punto)

Implemente las reglas de validación para la creación y la edición de una categoría. Se le proporciona el fichero `/src/controllers/Validation/ProductCategoryValidation.js` para definir estas validaciones.

```

const create = [
  check('name').exists().isString().isLength({ min: 1, max: 255 }).trim(),
  check('restaurantId').exists().isInt({ min: 1 }).toInt(),
  check('restaurantId').custom(checkRestaurantExists)
]

const update = [
  check('name').exists().isString().isLength({ min: 1, max: 255 }).trim(),
  check('restaurantId').exists().isInt({ min: 1 }).toInt(),
  check('restaurantId').exists().custom(checkRestaurantExists)
]

```



Para los que no esperan al último día.
Eres el Asalto: rompes las líneas del temario con presión constante. Tu misión: Atravesar el bloque más difícil sin mirar atrás.

BATTLEFIELD 6 | REDESEC
TEMPORADA 2

Equípate con granadas de subrayadores y toma el control de tu clase.

JUEGA AHORA

4. Controlador de ProductCategory (3 puntos)

Implemente las funciones necesarias para RF1, RF2, RF3. Se le proporciona el fichero `/src/controllers/ProductCategoryController.js` para definir estas funciones.

```
const ProductCategoryController = {
  async index (req, res) {
    try {
      const productCategories = await ProductCategory.findAll({
        where: { restaurantId: req.params.restaurantId },
      })
      res.json(productCategories)
    } catch (err) {
      res.status(500).send(err)
    }
  },

  async create (req, res) {
    try {
      const newCategory = ProductCategory.build(req.body)
      newCategory.restaurantId = req.params.restaurantId
      const createdCategory = await newCategory.save()
      res.json(createdCategory)
    }
    catch(err) {
      res.status(500).send(err)
    }
  },

  async destroy (req, res) {
    try {
      const result = await ProductCategory.destroy({ where: { id: req.params.categoryId } })
      let message = ''
      if (result === 1) {
        message = 'Sucessfully deleted category id.' + req.params.restaurantId
      } else {
        message = 'Could not delete category.'
      }
      res.json(message)
    } catch (err) {
      res.status(500).send(err)
    }
  }
}
```



WUOLAH

5. Comprobación de que el owner puede borrar una Categoría de producto (2 puntos)

Implemente la comprobación que un owner debe cumplir para borrar una categoría de producto, esto es:

- No hay ningún producto que tenga esa categoría asignada.
- El restaurante asociado a la categoría del producto es propiedad del usuario que realiza la petición.

Se le proporciona el fichero `/src/middlewares/ProductCategoryMiddleware.js` con el prototipo de las funciones `checkNoProductsInCategory` y `checkProductCategoryOwnership` que ha de implementar.

```
const checkProductCategoryOwnership = async (req, res, next) => {
  try {
    const category = await ProductCategory.findByPk(req.params.productCategoryId, {
      include: {
        model: Restaurant,
        attributes: ['id', 'userId']
      }
    })
    if (!category) {
      return res.status(404).json({ error: 'Category not found' })
    }
    if (category.Restaurant.userId !== req.user.id) {
      return res.status(403).json({ error: 'Forbidden' })
    }
    next()
  }
  catch (err){
    return res.status(500).send(err)
  }
}

// No debe haber ningún producto que tenga la categoría asignada
const checkNoProductsInCategory = async (req, res, next) => {
  try {
    const productosConCategoria = await Product.count({
      where: { productCategoryId: req.params.productCategoryId }
    })

    if (productosConCategoria > 0) {
      throw new Error("Yoy can not delete a category with associated products");
    }
    next()
  }
  catch (err){
    return res.status(500).send(err)
  }
}
```